

AFRILANG Stdlib

std/c/mlcosine

Français. Bibliothèque AFRILANG 'std/c/mlcosine' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : cosineSim, cosineDist, normalize, angularDist, batchCosine, mostSimilar.

```
'import "std/c/mlcosine"' · 'use mlcosine'
```

- 'cosineSim(a list number, b list number) → number' - 'cosineDist(a list number, b list number) → number' - 'normalize(v list number) → list number' - 'angularDist(a list number, b list number) → number' - 'batchCosine(query list number, matrix list number, dims number) → list number' - 'mostSimilar(query list number, matrix list number, dims number) → number'

English. AFRILANG library 'std/c/mlcosine' (tier complex, category complex). Exports 6 function(s): cosineSim, cosineDist, normalize, angularDist, batchCosine, mostSimilar.

```
'import "std/c/mlcosine"' · 'use mlcosine'
```

- 'cosineSim(a list number, b list number) → number' - 'cosineDist(a list number, b list number) → number' - 'normalize(v list number) → list number' - 'angularDist(a list number, b list number) → number' - 'batchCosine(query list number, matrix list number, dims number) → list number' - 'mostSimilar(query list number, matrix list number, dims number) → number'

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	6	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/mlcosine' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : cosineSim, cosineDist, normalize, angularDist, batchCosine, mostSimilar.

```
'import "std/c/mlcosine"' · 'use mlcosine'
```

```
- `cosineSim(a list number, b list number) → number`
- `cosineDist(a list number, b list number) → number`
- `normalize(v list number) → list number`
- `angularDist(a list number, b list number) → number`
- `batchCosine(query list number, matrix list number, dims number) → list number`
- `mostSimilar(query list number, matrix list number, dims number) → number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/mlcosine"
use mlcosine
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- cosineSim(a list number, b list number) → number•
- cosineDist(a list number, b list number) → number•
- normalize(v list number) → list•
- angularDist(a list number, b list number) → number•
- batchCosine(query list number, matrix list number, dims number) → list•
- mostSimilar(query list number, matrix list number, dims number) → number

4. Exemples d'utilisation

```
import "std/c/mlcosine"
use mlcosine
```

```
create result = cosineSim(/* args */)
say result
```

```
create result = cosineDist(/* args */)
say result
```

```
create result = normalize(/* args */)
say result
```

```
create result = angularDist(/* args */)
say result
```

```
create result = batchCosine(/* args */)
say result
```

```
create result = mostSimilar(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/mlcosine"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module mlcosine

```
function cosineSim(a list number, b list number) returns number
end
```

```
function cosineDist(a list number, b list number) returns number
end
```

```
function normalize(v list number) returns list number
end
```

```
function angularDist(a list number, b list number) returns number
end
```

```
function batchCosine(query list number, matrix list number, dims number) returns list
    number
end
```

```
function mostSimilar(query list number, matrix list number, dims number) returns
    number
end
end
```

English

1. Overview

AFRILANG library 'std/c/mlcosine' (tier complex, category complex). Exports 6 function(s): cosineSim, cosineDist, normalize, angularDist, batchCosine, mostSimilar.

```
'import "std/c/mlcosine"' · 'use mlcosine'
```

```
- `cosineSim(a list number, b list number) → number`
- `cosineDist(a list number, b list number) → number`
- `normalize(v list number) → list number`
- `angularDist(a list number, b list number) → number`
- `batchCosine(query list number, matrix list number, dims number) → list number`
- `mostSimilar(query list number, matrix list number, dims number) → number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/mlcosine"
use mlcosine
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- cosineSim(a list number, b list number) → number•
- cosineDist(a list number, b list number) → number•
- normalize(v list number) → list•
- angularDist(a list number, b list number) → number•
- batchCosine(query list number, matrix list number, dims number) → list•
- mostSimilar(query list number, matrix list number, dims number) → number

4. Usage examples

```
import "std/c/mlcosine"
use mlcosine
```

```
create result = cosineSim(/* args */)
say result
```

```
create result = cosineDist(/* args */)
say result
```

```
create result = normalize(/* args */)
say result
```

```
create result = angularDist(/* args */)
say result
```

```
create result = batchCosine(/* args */)
say result
```

```
create result = mostSimilar(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/mlcosine"`` at the top of your file.
- Run ``afri-lang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module mlcosine

```
function cosineSim(a list number, b list number) returns number
end
```

```
function cosineDist(a list number, b list number) returns number
end
```

```
function normalize(v list number) returns list number
end
```

```
function angularDist(a list number, b list number) returns number
end
```

```
function batchCosine(query list number, matrix list number, dims number) returns list
  number
end
```

```
function mostSimilar(query list number, matrix list number, dims number) returns
  number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/mlcosine"`
- Vérification : `afri-lang check`
- Documentation : <https://afri-lang.dev/stdlib/std/c/mlcosine/>

Source (extrait)

```
module mlcosine

function cosineSim(a list number, b list number) returns number
end

function cosineDist(a list number, b list number) returns number
end

function normalize(v list number) returns list number
end

function angularDist(a list number, b list number) returns number
end
```

```
function batchCosine(query list number, matrix list number, dims number) returns list
    number
end

function mostSimilar(query list number, matrix list number, dims number) returns
    number
end
end
```